

Premium UI & Frontend Blueprint

A clean research-workbench UX, component structure, responsive behavior, accessibility, and re-render control.

Recommended build: Option 2 - ASK -> CLARIFY -> DEFINE -> TEST -> LEARN

Source basis: the complete 13-page assignment PDF provided by you. Page references inside these guides refer to that assignment.

1. UX principle: research workspace, not trading terminal

Use a premium but calm B2B/SaaS interface. The assignment says the most beautiful UI is not the primary target, so visual polish should support comprehension rather than dominate it.

Recommended visual system

Token	Recommendation	Use
Primary	Cyprus #004741	Headings, primary buttons, active stages
Background	Warm sand / off-white #F5F2EA	App canvas
Surface	White / very light neutral	Cards
Success	Muted green	Validated/confirmed fields
Warning	Soft amber	Assumptions / missing information
Danger	Muted brick	Validation/API errors
Typography	Inter / Geist / system sans	Readable product UI

Page anatomy

```

■ Research Assistant                                     Prototype ■
■ Ask → Clarify → Define → Test → Learn                ■
■ "What market idea do you want to investigate?"        ■
■ [ Does buying NIFTY after a sharp fall work?          ] [→] ■
■ example chips                                          ■
■ CLARIFY                                                ■
■ Sharp fall? [1%]   Hold? [5 days]   Test period? [10 years] ■
■ DEFINED EXPERIMENT                                     ■ WHY THIS MATTERS ■
■ Market: NIFTY                                         ■ User said / Assumed / Asked ■
■ Entry: ...                                             ■ provenance legend ■
■ Exit: ...                                              ■
■ [ Run sample test ]                                   ■
■ TEST / LEARN: metrics, distribution, evidence, conclusion ■

```

Responsive behavior

- Desktop: two-column Define stage (experiment + provenance/explanation).
- Tablet: single column, stage rail stays horizontal and scrollable.
- Mobile: sticky bottom action; cards stack; no wide tables.
- Keep input and primary action above the fold.
- Use skeletons only for network calls; do not animate every card.

2. Component architecture

src/

```

app/
  page.tsx
  api/research/parse/route.ts
  api/research/test/route.ts
components/research/
  ResearchWorkbench.tsx
  ResearchPrompt.tsx
  StageStepper.tsx
  ClarificationPanel.tsx
  ClarificationField.tsx
  ExperimentCard.tsx
  ProvenanceBadge.tsx
  TestPanel.tsx
  ResultSummary.tsx
  EvidenceVsConclusion.tsx
  NextQuestions.tsx
lib/
  schema/experiment.ts
  ai/parse-question.ts
  research/mock-backtest.ts
  research/metrics.ts
  security/rate-limit.ts
  utils.ts
types/research.ts

```

State ownership - avoid unnecessary re-renders

- Keep the canonical workflow state in one reducer or a small Zustand store, not dozens of unrelated useState calls.
- Store form draft locally inside the clarification form; commit to global state only on submit/change blur when appropriate.
- Use React.memo only for components with measurable prop-stability benefit; do not memoize everything blindly.
- Use useMemo for derived experiment summaries/metrics, not for trivial string operations.
- Use useCallback only when passing callbacks to memoized children or dependencies require stable identity.
- Prefer server components for static layout/content and client components only for interactive workflow.
- Debounce only network-bound free-text operations; do not debounce select/radio interactions.

Suggested reducer states

```

type Stage = 'ask' | 'clarify' | 'define' | 'test' | 'learn';

type ResearchState = {
  stage: Stage;
  query: string;
  experiment: Experiment | null;
  missingFields: MissingField[];
  result: BacktestResult | null;
  status: 'idle' | 'parsing' | 'testing' | 'error';
  error?: string;
};

```

3. Interaction details evaluators will notice

Moment	Good UX
Initial state	One primary question box, 2-3 examples, one sentence explaining the workflow.
Parsing	Show “Structuring your idea...” rather than pretending the AI is certain.
Missing info	Ask only high-impact questions; mark optional details separately.

Moment	Good UX
Assumption	Present a proposed value with "Assumption - confirm or edit".
Define	Allow editing before test; show source/provenance badge for each field.
Test	Explain sample/mock data and methodology; never imply live trading evidence if it is not.
Learn	Separate factual metric cards from AI interpretation.
Error	Keep user input, show retry, never clear the workflow.

Accessibility and polish

- Visible labels; placeholders are not labels.
- Keyboard navigation for all controls.
- Focus ring on interactive components.
- Minimum contrast appropriate for text/actions.
- aria-live for parse/test status messages.
- Never use color alone to distinguish User said / Assumed / Asked.

4. Frontend quality gate

- No hydration errors.
- No console errors/warnings.
- No layout shift when results load.
- No form reset on failed network request.
- Loading button cannot be double-submitted.
- Mobile at 360px works.
- Tab order is logical.
- All major empty/error/loading/success states exist.

Confirmed Implementation Stack

This page supersedes any earlier wording that presented the AI provider or database as optional choices. For this submission, the selected implementation is fixed:

Layer	Selected technology	How it is used
AI / LLM	Google Gemini API	Parse natural-language research questions, identify ambiguity, propose clarification questions
Database	Neon PostgreSQL	Persist research sessions, structured experiments, clarification answers, test runs, and con
ORM	Drizzle ORM	Typed schema, migrations, and database access from the Next.js server layer.
Validation	Zod	Validate both user input and Gemini structured output before persistence or testing.
Frontend / Backend	Next.js + TypeScript	App Router UI plus server-side route handlers/server functions.

Environment variables

```
GEMINI_API_KEY=...  
DATABASE_URL=postgresql://...  
GEMINI_MODEL=gemini-2.5-flash
```

Recommended data flow

Question -> server validation -> Gemini structured interpretation -> Zod validation -> clarification/confirmation -> experiment persisted to Neon -> deterministic TypeScript test -> result persisted -> Learn view.

Security rules

Never expose GEMINI_API_KEY or DATABASE_URL in client code. Keep both outside NEXT_PUBLIC variables. Validate request sizes, parse Gemini output against a strict schema, use parameterized ORM queries, add API rate limiting, and never let the LLM fabricate backtest metrics.

Suggested Neon schema

research_sessions: id, original_question, status, created_at
experiments: id, session_id, instrument, timeframe, entry_condition, exit_condition, holding_period, test_period, filters_json, costs_json, hypothesis, provenance_json
test_runs: id, experiment_id, dataset_source, metrics_json, warnings_json, created_at

Why this choice is strong for the assignment: Gemini is used for language understanding and ambiguity handling, while Neon stores reproducible research state. Numerical evidence remains deterministic and auditable instead of being invented by the model.